BUSINESS REQUIREMENTS DOCUMENT

Retail Sales Analytics

A structured SQL analytics initiative using PostgreSQL

Document Type	Business Requirements Document (BRD)
Project Domain	Retail Analytics / Business Intelligence
Technology Stack	PostgreSQL, pgAdmin 4, SQL
Dataset	Retail transaction records
Version	1.1 — Updated numbering format

Purpose: define the analytical objectives, business questions, data requirements, key performance indicators, and expected deliverables for a retail sales analysis project.

Prepared for portfolio documentation and demonstration of SQL-based business analysis.

1. Executive Summary

This project applies SQL-based analysis to retail transaction data to evaluate sales performance, customer purchasing patterns, product-category contribution, and transaction timing. PostgreSQL is used to query, aggregate, and validate the available records. The resulting analysis is intended to translate transactional data into clear, decision-supporting insights for retail stakeholders.

2. Business Context and Problem Statement

Retail teams require reliable visibility into what is selling, which customer groups are purchasing, which customers contribute the most sales, and when order activity is highest. Reviewing raw records individually is inefficient and can make patterns difficult to identify. This project addresses that need by applying structured SQL queries to consolidate transaction-level data into actionable summaries.

3. Project Objectives

• Quantify the total number of transactions and unique customers.

• Measure total sales and average transaction value using the available sales amount field.

• Compare revenue and transaction volume across product categories.

• Assess sales patterns by gender and age category where supported by the data.

• Evaluate monthly sales patterns and order volume by time-of-day segment.

• Identify the five customers with the highest cumulative sales.

• Assess data completeness by checking critical fields for NULL values.

• Identify transactions whose total sales exceed a defined threshold.

4. Scope

In Scope

• Exploratory review of retail transaction records.

• Data-quality checks for key identifiers, timestamps, customer attributes, quantities, and sales values.

• Descriptive analysis using filtering, aggregation, grouping, sorting, date/time extraction, CASE expressions, and CTEs.

• Documentation of business questions, query outputs, and evidence-based observations.

Out of Scope / Dependencies

• Profitability analysis beyond available fields and validated definitions.

• Customer retention analysis requiring longitudinal customer history and a clear definition of repeat activity.

• Causal claims about why sales change; these would require additional contextual data.

• Dashboard delivery unless a separate BI layer is added.

5. Dataset and Data Dictionary

The pgAdmin output shown during project development reports 500 rows. Confirm the final row count after any cleaning or filtering before publishing the final project documentation.

Column	Business Meaning
transaction_id	Identifier for a transaction; expected to distinguish individual transaction records.
sales_data	Product category label as named in the source dataset. Confirm spelling and meaning against the source.

Column	Business Meaning
sales_time	Transaction date and time, used for date-based and time-of-day analysis.
customer_id	Customer identifier used to count unique customers and aggregate customer sales.
gender	Customer gender category as recorded in the dataset.
age_category	Customer age group or category as recorded in the dataset.
quantity	Quantity associated with a transaction.
price_per_unit	Recorded price for one unit.
cogs	Cost of goods sold, subject to the source dataset's definition.
total_sales	Recorded sales amount used for revenue-oriented aggregation.

6. Business Requirements

1. Transaction Volume

Calculate the number of transaction records to establish the dataset's transaction volume.

2. Customer Reach

Count distinct customer identifiers to estimate the number of unique customers represented.

3. Sales Performance

Calculate total sales and average recorded sales amount using `total_sales`.

4. Category Performance

Rank product categories by total sales and transaction count to identify leading categories.

5. Customer Segment Analysis

Compare transaction volume and sales by gender and age category, where values are available.

6. Time-Based Analysis

Summarize sales by year and month, and classify transactions into time-of-day segments using `sales_time`.

7. Customer Contribution

Rank customers by cumulative sales and report the top five customers.

8. Data Quality

Identify records with missing values in critical fields and report the extent of missingness.

9. High-Value Transactions

Retrieve transactions where `total_sales` exceeds a defined threshold, such as 100.

10. Category Customer Reach

Count distinct customers represented within each product category.

7. Business Questions

1. How many transaction records are present in the dataset?

2. How many unique customers are represented?

3. Are any critical transaction or sales fields missing?

4. What is the total recorded sales amount?

5. Which product category generates the highest total sales?

6. Which product category has the highest number of transactions?

7. What is the average unit price for a selected category?

8. Which transactions have total sales greater than 100?

9. How does transaction volume vary by gender and product category?

10. What is the average sales amount for each year and month?

11. Which five customers have the highest cumulative sales?

12. How many unique customers are represented in each product category?

13. Which time-of-day segment has the highest order count?

8. Key Performance Indicators

KPI	Definition	Purpose
Total Transactions	Count of transaction records; use COUNT(*) when one row represents one transaction.	Measures recorded transaction volume.
Unique Customers	COUNT(DISTINCT customer_id).	Estimates customer reach in the dataset.
Total Sales	SUM(total_sales), after validating units and data quality.	Measures recorded sales value.
Average Sales Amount	AVG(total_sales).	Shows average sales value per record, if one row equals one transaction.
Category Transaction Count	Count of records grouped by product category.	Highlights category-level order activity.
Monthly Sales	SUM(total_sales) grouped by year and month.	Supports trend review over time.
Missing-Value Records	Count of records with one or more critical fields missing.	Highlights data quality concerns.

9. Analytical Approach and SQL Techniques

The project uses PostgreSQL queries to inspect records, filter transactions, aggregate numeric values, group data by category or customer, sort rankings, extract year/month/hour from timestamps, and create time-of-day labels with CASE expressions. A Common Table Expression (CTE) is used to make time-segment analysis easier to read and maintain.

Core SQL capabilities demonstrated:

• SELECT, WHERE, LIMIT, ORDER BY

• COUNT, COUNT(DISTINCT), SUM, AVG, ROUND

• GROUP BY and category/customer-level aggregation

• NULL checks and data-quality review

• EXTRACT for date/time analysis

• CASE WHEN for time-of-day classification

• CTEs using WITH

10. Expected Deliverables

• A well-organized SQL script containing commented business questions and executable queries.

• A documented data dictionary and data-quality assessment.

• Screenshots or exported results for selected queries.

• A GitHub README describing the project, technologies, questions, and findings.

• Optional Power BI dashboard as a future enhancement.

11. Assumptions, Risks, and Validation

• **Field naming:** The column is named `sales_data` in the supplied screenshot; verify whether it represents product category and preserve the actual schema name in SQL.

• **Transaction grain:** Confirm whether one row corresponds to one transaction before interpreting COUNT(*) as order count.

• **Sales definition:** Treat `total_sales` as recorded sales value; do not label it profit unless the business definition supports that interpretation.

• **Time segments:** The current logic labels hours before 12 as Morning, 12–17 as Afternoon, and all remaining hours as Evening.

• **Data completeness:** Validate row counts and NULL results after any data cleaning.

• **Privacy:** Avoid publishing sensitive customer-level information. Use anonymized identifiers and share only permitted dataset files.

12. Acceptance Criteria

• All documented SQL queries execute successfully against the stated PostgreSQL table schema.

• Business questions are clearly linked to the corresponding SQL queries.

• Aggregate results are checked against the underlying data and expected row counts.

• Dataset limitations and assumptions are documented rather than presented as confirmed facts.

• The GitHub repository contains the SQL script, BRD, README, and selected non-sensitive evidence.

13. Conclusion

This BRD establishes a practical framework for a SQL-based retail sales analytics project. By combining transaction, customer, category, and time-based analysis with data-quality checks, the project demonstrates how structured querying can support operational visibility and evidence-based business decisions. The findings should be finalized only after the relevant queries have been executed and their outputs reviewed.